算法设计与分析

递归与分治策略

方贤忠

宁波大学
计算机科学与技术系
fangxianzhong@nbu.edu.cn
https://xzfang.top

Spring & Summer, 2025



概要

- 基本思想
- 递归
- 递归案例
- 分治法
- 二分查找
- 大整数的乘法
- Strassen 矩阵乘法
- 棋盘覆盖
- 合并排序
- 快速排序
- 线性时间选择
- 最接近点对问题
- 循环赛日程表



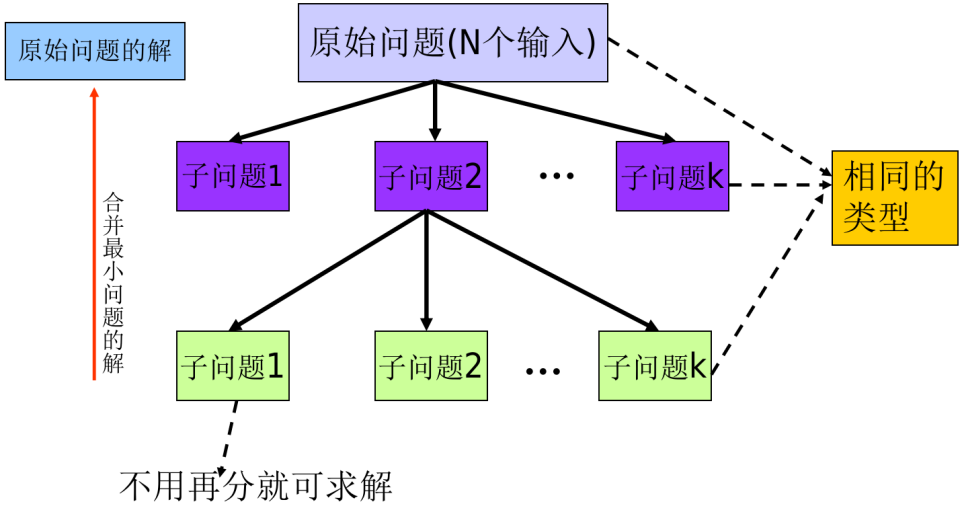
递归与分治策略的基本思想

分解-求解-合并 (Divide-Conquer-Combine)

- 将问题分解成 k 个子问题，对这 k 个子问题分别求解.
  - 如果子问题的规模仍然不够小，则再划分为 k 个子问题，如此递归的进行下去，直到问题规模足够小，很容易求出其解为止.
- 将求出的小规模的问题的解合并为一个更大规模的问题的解，自底向上逐步求出原来问题的解.



递归与分治策略的基本思想



递归

- 直接或间接地调用自身的算法称为递归算法.
- 用函数自身给出定义的函数称为递归函数.

由分治法产生的子问题往往是原问题的较小模式，这就为使用递归技术提供了方便。在这种情况下，反复应用分治手段，可以使子问题与原问题类型一致而其规模却不断缩小，最终使子问题缩小到很容易直接求出其解. 这自然导致递归过程的产生.

分治与递归像一对孪生兄弟，经常同时应用在算法设计之中，并由此产生许多高效算法.

阶乘函数

阶乘函数可递归地定义为:

$$n! = \begin{cases} 1 & n = 0 \quad \text{(边界条件)} \\ n(n-1)! & n > 0 \quad \text{(递归方程)} \end{cases}$$



阶乘函数

阶乘函数可递归地定义为:

$$n! = \begin{cases} 1 & n = 0 \quad \text{(边界条件)} \\ n(n-1)! & n > 0 \quad \text{(递归方程)} \end{cases}$$

边界条件 与递归方程 是递归函数的二个要素, 递归函数只有具备了这两个要素, 才能在有限次计算后得出结果.



Fibonacci 数列

无穷数列 $1, 1, 2, 3, 5, 8, 13, 21, 34, 55, \cdots$, **称为** Fibonacci **数列**.

Fibonacci **数列可递归地定义为**:

$$F(n) = \begin{cases} 1 & n = 0 \quad \text{(边界条件)} \\ 1 & n = 1 \quad \text{(边界条件)} \\ F(n-1) + F(n-2) & n > 1 \quad \text{(递归方程)} \end{cases}$$



Fibonacci 数列

无穷数列 $1, 1, 2, 3, 5, 8, 13, 21, 34, 55, \cdots$, **称为** Fibonacci **数列**.

Fibonacci **数列可递归地定义为**:

$$F(n) = \begin{cases} 1 & n = 0 \quad (\text{边界条件}) \\ 1 & n = 1 \quad (\text{边界条件}) \\ F(n-1) + F(n-2) & n > 1 \quad (\text{递归方程}) \end{cases}$$

```
1 int fibonacci(int n)
2 {
3   if (n <= 1) return 1;
4   return fibonacci(n-1) + fibonacci(n-2);
5 }
```

Ackerman 函数

当一个函数及它的一个变量是由函数自身定义时，称这个函数是双递归函数.

Ackerman 函数 $A(n,m)$ 定义如下:

$$A(n,m)=\begin{cases}2 & n=1,m=0\\ 1 & n=0,m\geq 0\\ n+2 & n\geq 2,m=0\\ A(A(n-1,m),m-1) & n\geq 1,m\geq 1\end{cases}$$

Ackerman 函数无法找到非递归的定义.



Ackerman 函数

当
是

Ac

Ac

$A(n, m)$ 的自变量 m 的每一个值都定义了一个单变量函数:

- $m = 0$ 时, $A(n, 0) = n + 2$;
- $m = 1$ 时, $A(n, 1) = A(A(n-1, 1), 0) = A(n-1, 1) + 2$, 和 $A(1, 1) = 2$, 故 $A(n, 1) = 2n$.
- $m = 2$ 时, $A(n, 2) = A(A(n-1, 2), 1) = 2A(n-1, 2)$, 和 $A(1, 2) = A(A(0, 2), 1) = A(1, 1) = 2$, 故 $A(n, 2) = 2^n$.
- $m = 3$ 时, 类似的可以推出

$$A(n, 3) = \underbrace{2^{2^{2^{\cdot^{\cdot^{\cdot}}}}}}_{n}.$$

- $m = 4$ 时, $A(n, 4)$ 的增长速度非常快, 以至于没有适当的数学式来表示.

排列问题

设计一个递归算法生成 n 个元素 $\{r_1, r_2, \ldots, r_n\}$ 的全排列.



排列问题

设计一个递归算法生成 n 个元素 $\{r_1, r_2, \ldots, r_n\}$ 的全排列.

设 $R = \{r_1, r_2, \ldots, r_n\}$ 是要进行排列的 n 个元素, $R_i = R - \{r_i\}$.
集合 X 中元素的全排列记为 $Perm(X)$.
R 的全排列可递归定义如下:

- 当 $n = 1$ 时, $Perm(R) = (r)$, 其中 r 是集合 R 中唯一的元素;
- 当 $n > 1$ 时, $Perm(R)$ 由 $(r_1)Perm(R_1), (r_2)Perm(R_2), \ldots, (r_n)Perm(R_n)$ 构成.



整数划分问题

将正整数 n 表示成一系列正整数之和: $n = n_1 + n_2 + \cdots + n_k$, 其中 $n_1 \geq n_2 \geq \cdots \geq n_k \geq 1$, $k \geq 1$. 求正整数 n 的不同划分个数.



整数划分问题

将正整数 n 表示成一系列正整数之和: $n = n_1 + n_2 + \cdots + n_k$, 其中 $n_1 \geq n_2 \geq \cdots \geq n_k \geq 1$, $k \geq 1$. 求正整数 n 的不同划分个数.

例如正整数 6 有如下 11 种不同的划分:

6;

5+1;

4+2, 4+1+1;

3+3, 3+2+1, 3+1+1+1;

2+2+2, 2+2+1+1, 2+1+1+1+1;

1+1+1+1+1+1.



整数划分问题

将正整数 n 表示成一系列正整数之和: $n = n_1 + n_2 + \cdots + n_k$, 其中 $n_1 \geq n_2 \geq \cdots \geq n_k \geq 1$, $k \geq 1$. 求正整数 n 的不同划分个数.

定义正整数 n 划分后最大数不超过 m 的划分个数为 $D(n, m)$, 则 $D(n, m)$ 可以分为两大类:

- $D(n - m, m)$, 即划分中至少含有一个 m;
- $D(n, m - 1)$, 即划分中最大值小于 m.

从而可以建立递归方程. 另外需要考虑边界条件:

- $D(n, 1) = 1, n > 0$;
- $D(1, m) = 1, m > 0$;
- $D(n, n) = 1 + D(n, n - 1)$.

整数划分问题

将正整数 n 表示成一系列正整数之和: $n = n_1 + n_2 + \cdots + n_k$, 其中 $n_1 \geq n_2 \geq \cdots \geq n_k \geq 1$, $k \geq 1$. 求正整数 n 的不同划分个数.

定义正整数 n 划分后最大数字不超过 m 的划分个数为 $D(n, m)$, 则

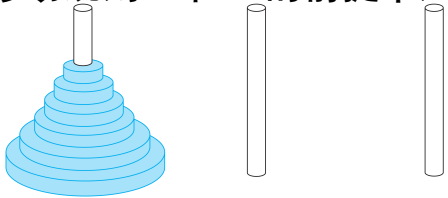
$$D(n,m) = \begin{cases} 1 & n = 0, 1 \\ 1 & m = 1 \\ D(n,n) & n < m \\ D(n-m,m) + D(n,m-1) & n, m > 1 \end{cases}$$



Hanoi 塔问题

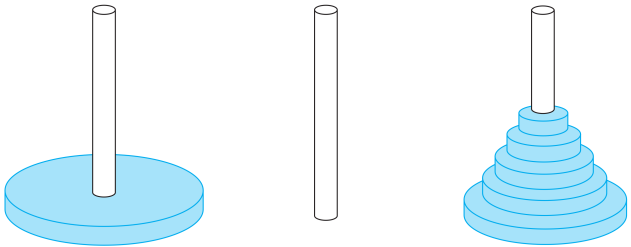
设 a,b,c 是 3 个塔座. 开始时, 在塔座 a 上有一叠共 n 个圆盘, 这些圆盘自下而上, 由大到小地叠在一起. 各圆盘从小到大编号为 $1, 2, \ldots, n$, 现要求将塔座 a 上的这一叠圆盘移到塔座 b 上, 并仍按同样顺序叠置. 移动圆盘时应遵守以下规则:

- 规则 1: 每次只能移动 1 个圆盘;
- 规则 2: 任何时刻都不允许将较大的圆盘压在较小的圆盘之上;
- 规则 3: 在满足移动规则 1 和 2 的前提下, 可将圆盘移至 a,b,c 中任一塔座上.



Hanoi 塔问题

```
1 void hanoi(int n, char a, char b, char c)
2 {
3   if (n > 0) {
4     hanoi(n-1, a, c, b); //move n-1 disks from a to c
5     move(a,b);           //move the last one from a to b
6     hanoi(n-1, c, b, a); //move n-1 disks from c to b
7   }
8 }
```



分治法

分治法所能解决的问题一般具有以下几个特征:

- 该问题的规模缩小到一定的程度就可以容易地解决;
- 该问题可以分解为若干个规模较小的相同问题，即该问题具有最优子结构性质;
- 利用该问题分解出的子问题的解可以合并为该问题的解;
- 该问题所分解出的各个子问题是相互独立的，即子问题之间不包含公共的子问题.



分治法

分治法所能解决的问题一般具有以下几个特征:

- 该问题的规模缩小到一定的程度就可以容易地解决;
- 该问题可以分解为若干个规模较小的相同问题，即该问题具有最优子结构
- 利用该问题
- 该问题所分

题的解;

子问题之间不

包含公共的

最后一条特征涉及到分治法的效率，如果各子问题是不独立的，则分治法要做许多不必要的工作，重复地解公共的子问题，此时虽然也可用分治法，但一般用动态规划较好.



分治法

```
1 divide_and_conquer(P)
2 {
3   if (|P| <= n0) return adhoc(P);           //解决小规模的问题
4   divide P into smaller subinstances {P1,P2,..,Pk};//分解问题
5   for (i=1; i<=k; ++i)                      //参考全排列的子问题
6     yi = divide_and_conquer(Pi);             //递归求解子问题
7   return merge(y1,...,yk);    //将各子问题的解合并为原问题的解
8 }
```

用分治法设计算法时, 最好使子问题的规模大致相同, 即将一个问题分成大小相等的 k 个子问题的处理方法是行之有效的. 使子问题规模大致相等的做法是出自一种平衡 (balancing) 子问题的思想, 它几乎总是比不平衡的做法好.



分治法的复杂性分析

一个分治法将规模为 n 的问题分成 k 个规模为 n/m 的子问题去解. 设分解阀值 $n_0 = 1$，且 adhoc 解规模为 1 的问题耗费 1 个单位时间. 再设将原问题分解为 k 个子问题以及用 merge 将 k 个子问题的解合并为原问题的解需用 $f(n)$ 个单位时间. 用 $T(n)$ 表示该分治法解规模为 $|P| = n$ 的问题所需的计算时间，则有:

$$T(n) = \begin{cases} O(1) & n = 1 \\ kT(n/m) + f(n) & n > 1 \end{cases}$$



分治法的复杂性分析

一个分治法将规模为 n 的问题分成 k 个规模为 n/m 的子问题去解. 设分解阀值 $n_0 = 1$, 且 adhoc 解规模为 1 的问题耗费 1 个单位时间. 再设将原问题分解为 k 个子问题以及用 merge 将 k 个子问题的解合并为原问题的解需用 $f(n)$ 个单位时间. 用 $T(n)$ 表示该分治法解规模为 $|P| = n$ 的问题所需的计算时间, 则有:

$$T(n) = \begin{cases} O(1) & n = 1 \\ kT(n/m) + f(n) & n > 1 \end{cases}$$

通过迭代法求得方程的解:

$$T(n) = n^{\log_m k} + \sum_{j=0}^{\log_m n - 1} k^j f(n/m^j).$$

分治法的复杂性分析

一个分治法将规模为 n 的问题分成 k 个规模为 n/m 的子问题去解. 设分解阀值 $n_0 = 1$ 且 $n = n_0$ 时规模为 1 的问题耗费 1 个单位时间. 再设将原问题分解为 k 个子问题以及将 k 个子问题的解合并为原问题的解需用 $f(n)$ 个单位时间. 用 $T(n)$ 表示该分治法解规模为 $|P| = n$ 的问题所需的计算时间, 则有:

$$T(n) = \begin{cases} 1 & n = 1 \\ kT(n/m) + f(n) & n > 1 \end{cases}$$

通过迭代法求

注意:

- 递归方程及其解只给出 n 等于 m 的方幂时 $T(n)$ 的值, 但是如果认为 $T(n)$ 足够平滑, 那么由 n 等于 m 的方幂时 $T(n)$ 的值可以估计 $T(n)$ 的增长速度.
- 通常假定 $T(n)$ 是单调上升的, 从而当 $m_i \le n < m_{i+1}$ 时, $T(m_i) \le T(n) < T(m_{i+1})$.

二分查找

```
1  // input: A[0:m-1], increasing order
2  // output: find the index of element c
3  int bisearch(int m, const int* A, int c)
4  {
5    int l(0), u(m-1), i;
6
7    while (l<=u) {
8      i = (l+u)/2;
9      if (A[i]==c) {
10       return i; // find
11     } else if (A[i]>c) {
12       u = i-1;
13     } else {
14       l = i+1;
15     }//if
16   }//while
17
18   return -1;
19 }
```

算法复杂度分析

每执行一次算法的 while 循环，待搜索数组的大小减少一半. 因此，在最坏情况下，while 循环被执行了 $O(\log n)$ 次. 循环体内运算需要 $O(1)$ 时间，因此整个算法在最坏情况下的计算时间复杂性为 $O(\log n)$.



二分查找

查询 70

0	1	2	3	4	5	6	7	8	9	10	11	12
3	14	27	31	39	42	55	70	74	81	85	93	98
l						*m*						*r*
							l		*m*			*r*
							l,*m*	*r*				

大整数的乘法

设计一个有效的算法, 可以进行两个 n 位大整数的乘法运算.

- 小学的方法: $O(n^2)$;
- 分治法: X, Y 是两个 n 位二进制数,
  - $X = a \cdot 2^{n/2} + b$, $Y = c \cdot 2^{n/2} + d$;
  - $X \cdot Y = ac2^n + (ad + bc)2^{n/2} + bd$.



大整数的乘法

设计一个有效的算法, 可以进行两个 n 位大整数的乘法运算.

- 小学的方法: $O(n^2)$; 效率太低
- 分治法: X, Y 是两个 n 位二进制数,
  - $X = a \cdot 2^{n/2} + b$, $Y = c \cdot 2^{n/2} + d$;
  - $X \cdot Y = ac2^n + (ad + bc)2^{n/2} + bd$.



大整数的乘法

设计一个有效的算法, 可以进行两个 n 位大整数的乘法运算.

- 小学的方法: $O(n^2)$; 效率太低
- 分治法: X, Y 是两个 n 位二进制数.

复杂度分析:

$$T(n) = \begin{cases} O(1) & n = 1 \\ 4T(n/2) + O(n) & n > 1 \end{cases}$$

故 $T(n) = O(n^2)$.

大整数的乘法

设计一个有效的算法, 可以进行两个 n 位大整数的乘法运算.

- 小学的方法: $O(n^2)$; 效率太低
- 分治法: X, Y 是两个 n 位二进制数.

复杂度分析:

没有改进

$$T(n) = \begin{cases} O(1) & n = 1 \\ 4T(n/2) + O(n) & n > 1 \end{cases}$$

故 $T(n) = O(n^2)$.



大整数的乘法

为了降低时间复杂度, 必须减少乘法的次数.



大整数的乘法

为了降低时间复杂度，必须减少乘法的次数.

$$X \cdot Y = ac2^n + ((a-c)(b-d) + ac + bd)2^{n/2} + bd$$
$$X \cdot Y = ac2^n + ((a+c)(b+d) - ac - bd)2^{n/2} + bd$$



大整数的乘法

为了降低时间复杂度, 必须减少乘法的次数.

$$X \cdot Y = ac2^n + ((a-c)(b-d) + ac + bd)2^{n/2} + bd$$
$$X \cdot Y = ac2^n + ((a+c)(b+d) - ac - bd)2^{n/2} + bd$$

复杂度分析:

$$T(n) = \begin{cases} O(1) & n = 1 \\ 3T(n/2) + O(n) & n > 1 \end{cases}$$

故 $T(n) = O(n^{\log 3}) = O(n^{1.585})$.

大整数的乘法

为了降低时间复杂度, 必须减少乘法的次数.

$$X \cdot Y = ac2^n + ((a-c)(b-d) + ac + bd)2^{n/2} + bd$$
$$X \cdot Y = ac2^n + ((a+c)(b+d) - ac - bd)2^{n/2} + bd$$

复杂度分析:

改进较大

$$T(n) = \begin{cases} O(1) & n = 1 \\ 3T(n/2) + O(n) & n > 1 \end{cases}$$

故 $T(n) = O(n^{\log 3}) = O(n^{1.585})$.

大整数的乘法

为了降低时间复杂度, 必须减少乘法的次数.

$$X \cdot Y = ac2^n + ((a-c)(b-d) + ac + bd)2^{n/2} + bd$$
$$X \cdot Y = ac2^n + ((a+c)(b+d) - ac - bd)2^{n/2} + bd$$

两个 $X \cdot Y$ 的复杂度都是 $O(n^{\log 3})$,
但考虑到 $a + c$, $b + d$ 可能得到
$m + 1$ 位的结果, 使问题的规模变大.

复杂

$$\begin{cases} 3T(n/2) + O(n) & n > 1 \end{cases}$$

故 $T(n) = O(n^{\log 3}) = O(n^{1.585})$.

大整数的乘法

- 如果将大整数分成更多段，用更复杂的方式把它们组合起来，将有可能得到更优的算法.
- 最终，这个思想导致了快速傅利叶变换 (Fast Fourier Transform) 的产生. 该方法也可以看作是一个复杂的分治算法.



Strassen 矩阵乘法

矩阵 A 和 B 的乘积矩阵 C 中的元素 $C[i,j]$ 定义为:

$$C[i,j] = \sum_{k=1}^{n} A[i,k]B[k,j].$$

Strassen 矩阵乘法

矩阵 A 和 B 的乘积矩阵 C 中的元素 $C[i,j]$ 定义为:

$$C[i,j] = \sum_{k=1}^{n} A[i,k]B[k,j].$$

若依此定义来计算 A 和 B 的乘积矩阵 C, 则每计算 C 的一个元素 $C[i,j]$, 需要做 n 次乘法和 $n-1$ 次加法. 因此, 算出矩阵 C 的每个元素所需的计算时间为 $O(n^3)$.



Strassen 矩阵乘法

使用与上例类似的技术, 将矩阵 A, B, C 中每一矩阵都分块成 4 个大小相等的子矩阵. 由此可将方程 $C = AB$ 重写为:

$$\begin{bmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{bmatrix} = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix} \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix}$$



Strassen 矩阵乘法

使用与上例类似的技术, 将矩阵 A, B, C 中每一矩阵都分块成 4 个大小相等的子矩阵. 由此可将方程 $C = AB$ 重写为:

$$\begin{bmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{bmatrix} = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix} \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix}$$

$$\begin{aligned} C_{11} &= A_{11}B_{11} + A_{12}B_{21} \\ C_{12} &= A_{11}B_{12} + A_{12}B_{22} \\ C_{21} &= A_{21}B_{11} + A_{22}B_{21} \\ C_{22} &= A_{21}B_{12} + A_{22}B_{22} \end{aligned}$$

Strassen 矩阵乘法

使用与上例类似的技术, 将矩阵 A, B, C 中每一矩阵都分块成 4 个大小相等的子矩阵. 由此可将方程 $C = AB$ 重写为:

$$\begin{bmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{bmatrix} = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix} \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix}$$

复杂度分析:

$$T(n) = \begin{cases} O(1) & n = 1 \\ 8T(n/2) + O(n^2) & n > 1 \end{cases}$$

故 $T(n) = O(n^3)$.

Strassen 矩阵乘法

为了降低时间复杂度, 必须减少乘法的次数.

$$
\begin{aligned}
S_1 &= B_{12} - B_{22} \\
S_2 &= A_{11} + A_{12} \\
S_3 &= A_{21} + A_{22} \\
S_4 &= B_{21} - B_{11} \\
S_5 &= A_{11} + A_{22} \\
S_6 &= B_{11} + B_{22} \\
S_7 &= A_{12} - A_{22} \\
S_8 &= B_{21} + B_{22} \\
S_9 &= A_{11} - A_{21} \\
S_{10} &= B_{11} + B_{12}
\end{aligned}
\qquad
\begin{aligned}
P_1 &= A_{11}S_1 \\
P_2 &= S_2B_{22} \\
P_3 &= S_3B_{11} \\
P_4 &= A_{22}S_4 \\
P_5 &= S_5S_6 \\
P_6 &= S_7S_8 \\
P_7 &= S_9S_{10}
\end{aligned}
\qquad
\begin{aligned}
C_{11} &= P_5 + P_4 - P_2 + P_6 \\
C_{12} &= P_1 + P_2 \\
C_{21} &= P_3 + P_4 \\
C_{22} &= P_5 + P_1 - P_3 - P_7
\end{aligned}
$$



Strassen 矩阵乘法

为了降低时间复杂度, 必须减少乘法的次数.

$S_1 = B_{12} - B_{22}$ $P_1 = A_{11}S_1$ $C_{11} = P_5 + P_4 - P_2 + P_6$

S_2

S_3

S_4 $P_3 - P_7$

S_5

S_6

S_7

$S_8 = B_{21} + B_{22}$

$S_9 = A_{11} - A_{21}$

$S_{10} = B_{11} + B_{12}$

复杂度分析:

$$T(n) = \begin{cases} O(1) & n = 1 \\ 7T(n/2) + O(n^2) & n > 1 \end{cases}$$

故 $T(n) = O(n^{\log 7})$.

Strassen 矩阵乘法

- 传统方法: $O(n^3)$;
- 分治法: $O(n^{\log 7})$;
- 更快的方法?



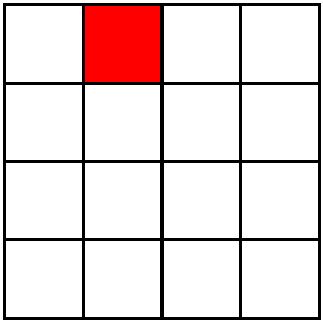
Strassen 矩阵乘法

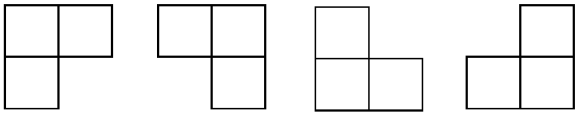
- 传统方法: $O(n^3)$;
- 分治法: $O(n^{\log 7})$;
- 更快的方法?

- Hopcroft 和 Kerr 已经证明 (1971), 计算 2 个 2×2 矩阵的乘积, 7 次乘法是必要的. 因此, 要想进一步改进矩阵乘法的时间复杂性, 就不能再基于计算 2×2 矩阵的 7 次乘法这样的方法了. 或许应当研究 3×3 或 5×5 矩阵的更好算法.
- 在 Strassen 之后又有许多算法改进了矩阵乘法的计算时间复杂性, 目前最好的计算时间上界是 $O(n^{2.376})$.
- 是否能找到 $O(n^2)$ 的算法?

棋盘覆盖

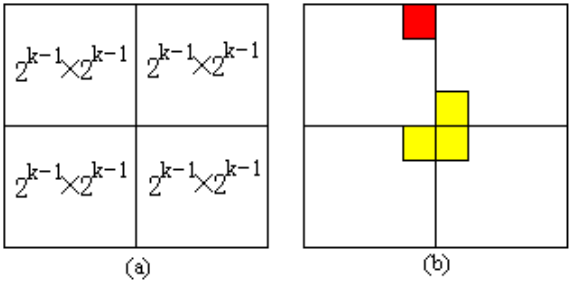
在一个 $2^k \times 2^k$ 个方格组成的棋盘中，恰有一个方格与其它方格不同，称该方格为一特殊方格，且称该棋盘为一特殊棋盘. 在棋盘覆盖问题中，要用图示的 4 种不同形态的 L 型骨牌覆盖给定的特殊棋盘上除特殊方格以外的所有方格，且任何 2 个 L 型骨牌不得重叠覆盖.





棋盘覆盖

当 $k > 0$ 时, 将 $2^k \times 2^k$ 棋盘分割为 4 个 $2^{k-1} \times 2^{k-1}$ 子棋盘 (a) 所示. 特殊方格必位于 4 个较小子棋盘之一中, 其余 3 个子棋盘中无特殊方格. 为了将这 3 个无特殊方格的子棋盘转化为特殊棋盘, 可以用一个 L 型骨牌覆盖这 3 个较小棋盘的会合处, 如 (b) 所示, 从而将原问题转化为 4 个较小规模的棋盘覆盖问题. 递归地使用这种分割, 直至棋盘简化为棋盘 1×1.



棋盘覆盖

当 $k > 0$ 时, 将 $2^k \times 2^k$ 棋盘分割为 4 个 $2^{k-1} \times 2^{k-1}$ 子棋盘 (a) 所示. 特殊方格必位于 4 个较小子棋盘之一中, 其余 3 个子棋盘中无特殊方格. 为了将这 3 个无特殊方格的子棋盘转化为特殊棋盘, 可以用一个 L 型骨牌覆盖这 3 个较小棋盘的会合处, 如 (b) 所示, 从而将原问题转化为 4 个较小规模的棋盘覆盖问题. 递归地使用这种分割, 直至棋盘简化为棋盘 1×1.

复杂度分析:

$$T(n) = \begin{cases} O(1) & n = 1 \\ 4T(n/2) + O(1) & n > 1 \end{cases}$$

故 $T(n) = O(n^2)$, $n = 2^k$.

(a) (b)

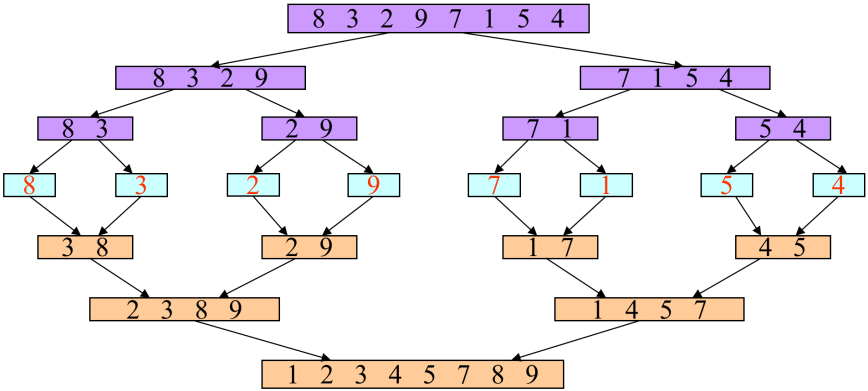
合并排序

将待排序元素分成大小大致相同的 2 个子集合，分别对 2 个子集合进行排序，最终将排好序的子集合合并成为所要求的排好序的集合.



合并排序

将待排序元素分成大小大致相同的 2 个子集合，分别对 2 个子集合进行排序，最终将排好序的子集合合并成为所要求的排好序的集合.



合并排序

```
1 MergeSort(A, p, r) :
2     if p < r then
3         q = (p + r)/2
4         MergeSort(A, p, q)
5         MergeSort(A, q + 1, r)
6         Merge(A, p, q, r)
7     end
8 end
```



合并排序

```
1  void merge(int* array, int p, int q, int r)
2  {
3    int* temp = new int [r-p+1];
4    int begin1(p), end1(q), begin2(q+1), end2(r), k(0);
5
6    while ( (begin1 <= end1) and ( begin2 <= end2) ) {  // 合并
7      if( array[begin1] < array[begin2]) {
8        temp[k++] = array[begin1++];
9      } else {
10       temp[k++] = array[begin2++];
11     }
12   }
13   while ( begin1 <= end1 ) {//若第一个序列有剩余，直接拷贝出来粘到合并序列尾
14     temp[k++] = array[begin1++];
15   }
16   while ( begin2 <= end2 ) {//若第二个序列有剩余，直接拷贝出来粘到合并序列尾
17     temp[k++] = array[begin2++];
18   }
19   for (i=0; i < r-p+1; ++i)                    //将排序好的序列拷贝回数组中
20     array[p+i] = temp[i];
21   delete[] temp;
22 }
```

合并排序

```
1  void merge(int* array, int p, int q, in
2  {
3    int* temp = new int [r-p+1];
4    int begin1(p), end1(q), begin2(q+1),
5
6    while ( (begin1 <= end1) and ( begin2
7      if( array[begin1] < array[begin2])
8        temp[k++] = array[begin1++];
9      } else {
10       temp[k++] = array[begin2++];
11     }
12   }
13   while ( begin1 <= end1 ) {//若第一个序列有剩余，直接拷贝出来粘到合并序列尾
14     temp[k++] = array[begin1++];
15   }
16   while ( begin2 <= end2 ) {//若第二个序列有剩余，直接拷贝出来粘到合并序列尾
17     temp[k++] = array[begin2++];
18   }
19   for (i=0; i < r-p+1; ++i)                    //将排序好的序列拷贝回数组中
20     array[p+i] = temp[i];
21   delete[] temp;
22 }
```

复杂度分析:

$$T(n) = \begin{cases} O(1) & n = 1 \\ 2T(n/2) + O(n) & n > 1 \end{cases}$$

故 $T(n) = O(n \log n)$.

合并排序

```
1  void merge(int* array, int p, int q, in
2  {
3    int* temp = new int [r-p+1];
4    int begin1(p), end1(q), begin2(q+1),
5
6    while ( (begin1 <= end1) and ( begin2
7      if( array[begin1] < array[begin2])
8        temp[k++] = array[begin1++];
9      } else {
10       temp[k++] = array[begin2++];
11     }
12   }
13   while ( begin1 <= end1 ) {//若第一个序列有剩余，直接拷贝出来粘到合并序列尾
14     temp[k++] = array[begin1++];
15   }
16   while ( begin2 <= end2 ) {//若第二个序
17     temp[k++] = array[begin2++];
18   }
19   for (i=0; i < r-p+1; ++i)
20     array[p+i] = temp[i];
21   delete[] temp;
22 }
```

复杂度分析:

$$T(n) = \begin{cases} O(1) & n = 1 \\ 2T(n/2) + O(n) & n > 1 \end{cases}$$

故 $T(n) = O(n \log n)$.

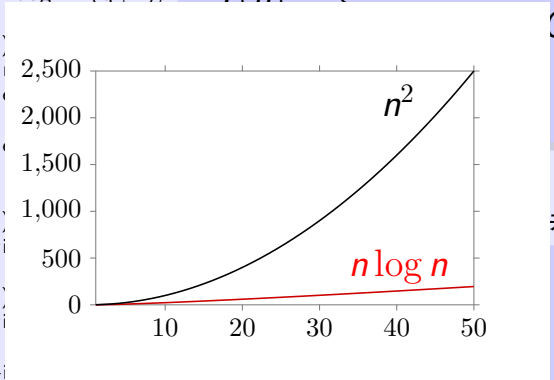
- **最坏时间复杂度**: $O(n \log n)$;
- **平均时间复杂度**: $O(n \log n)$;
- **辅助空间**: $O(n)$.

合并排序

```
1  void merge(int* array, int p, int q, in
2  {
3    int* temp = new int [r-p+1];
4    int begin1(p), end1(q), begin2(q+1),
5
6    while ( (begin1 <= end1)
7      if( array[begin1] < a
8        temp[k++] = array[b
9      } else {
10       temp[k++] = array[b
11     }
12   }
13   while ( begin1 <= end1 )
14     temp[k++] = array[begi
15   }
16   while ( begin2 <= end2 )
17     temp[k++] = array[begi
18   }
19   for (i=0; i < r-p+1; ++i
20     array[p+i] = temp[i];
21   delete[] temp;
22 }
```

复杂度分析:

$$T(n) = \begin{cases} O(1) & n = 1 \\ O(n) & n > 1 \end{cases}$$



并序列尾

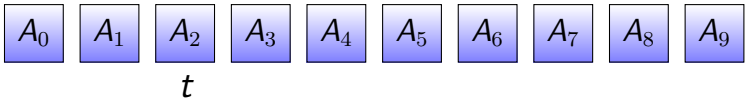
$O(n \log n)$;
$O(n \log n)$;

- 辅助空间: $O(n)$.

快速排序

基本思想

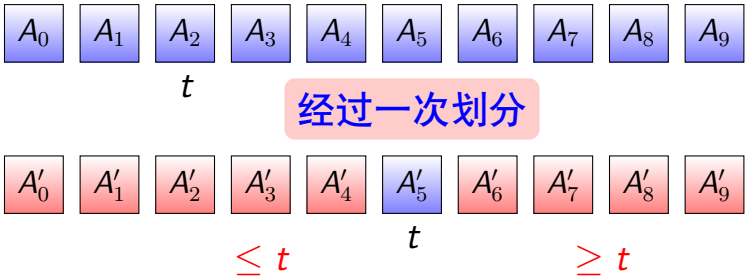
- 选取 A 的某个元素 $t = A[s]$, 然后将其他元素重新排列,
- 使得所有在 t 之前的元素都小于或等于 t,
- 而在 t 之后出现的元素都大于或等于 t.



快速排序

基本思想

- 选取 A 的某个元素 $t = A[s]$, 然后将其他元素重新排列,
- 使得所有在 t 之前的元素都小于或等于 t,
- 而在 t 之后出现的元素都大于或等于 t.



快速排序

快速排序（Quicksort）是对冒泡排序 的一种改进，
由 C. A. R. Hoare 在 1962 年提出.

基本思想

- 通过一趟排序将要排序的数据分割成独立的两部分，其中一部分的所有数据都比另外一部分的所有数据都要小；
- 然后再按此方法对这两部分数据分别进行快速排序，整个排序过程可以递归进行，以此达到整个数据变成有序序列.



快速排序

```
1 QuickSort(A, p, r) :
2     if p < r then
3         q =Partition(A, p, r)
4         QuickSort(A, p, q)
5         QuickSort(A, q + 1, r)
6     end
7 end
```



快速排序

```
1 QuickSort(A, p, r) :
2     if p < r then
3         q =Partition(A, p, r)
4         QuickSort(A, p, q)
5         QuickSort(A, q + 1, r)
6     end
7 end
```

划分元素选取 (Pivot)

- 可以随机选取, 最简单的选择策略是数组第一个元素;
- 如何得到一个较好效率的划分元素?

快速排序

划分的实现思想

同时从左、右开始扫描, 左边找到一个比划分元素大的, 右边找到一个比划分元素小的, 然后交换两个元素的位置.

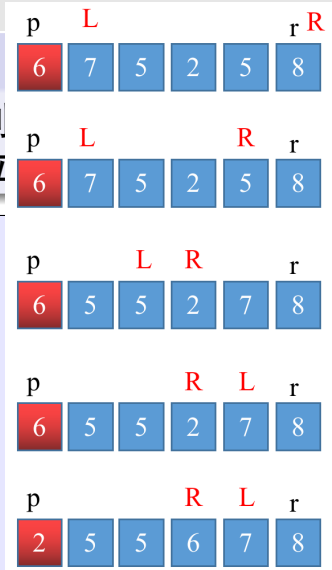
```
1  int partition(int* A, int p, int r)
2  {
3    int L(p+1), R(r), c(A[p]);
4    while (L <= R) {
5      if (L<r and A[L] <= c) ++L;
6      if (R>p and A[R] > c) --R;
7      if (L<R and A[L] > c and A[R] <= c) {
8        swap(A[L], A[R]);
9        ++L; --R;
10     }
11   }
12   swap(A[p], A[R]);
13   return R;
14 }
```

快速排序

划分的实现思想

同时从左、右开始扫描，左边找到一个比划
个比划分元素小的，然后交换两个元素的位

到一




```
1  int partition(int* A, int p, int r)
2  {
3    int L(p+1), R(r), c(A[p]);
4    while (L <= R) {
5      if (L<r and A[L] <= c) ++L;
6      if (R>p and A[R] > c) --R;
7      if (L<R and A[L] > c and A[R] <= c) {
8        swap(A[L], A[R]);
9        ++L; --R;
10     }
11   }
12   swap(A[p], A[R]);
13   return R;
14 }
```

快速排序

划分的实现思想 (第二种思路)

从一个方向 (例如从左向右) 逐次扫描元素.

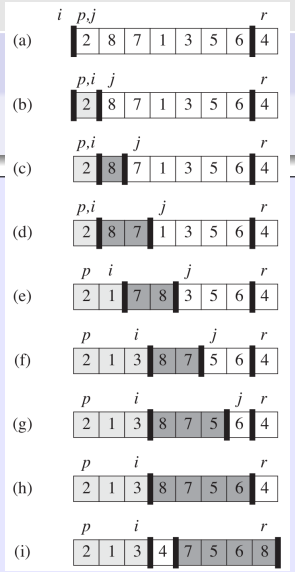
```
1  void partition(int* A, int p, int r)
2  {
3    int x = A[r];
4    int i = p-1;
5
6    for (int j=p; j < r; ++j) {
7      if (A[j] <= x) {
8        ++i;
9        swap(A[i], A[j]);
10     }//if
11   }//for
12
13   swap(A[i+1], A[r]);
14
15   return i+1;
16 }
```

快速排序

划分的实现思想 (第二种思路)

从一个方向 (例如从左向右) 逐次扫描元素.

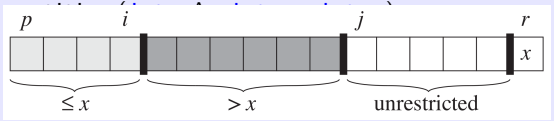
```
1  void partition(int* A, int p, int r)
2  {
3    int x = A[r];
4    int i = p-1;
5
6    for (int j=p; j < r; ++j) {
7      if (A[j] <= x) {
8        ++i;
9        swap(A[i], A[j]);
10     }//if
11   }//for
12
13   swap(A[i+1], A[r]);
14
15   return i+1;
16 }
```



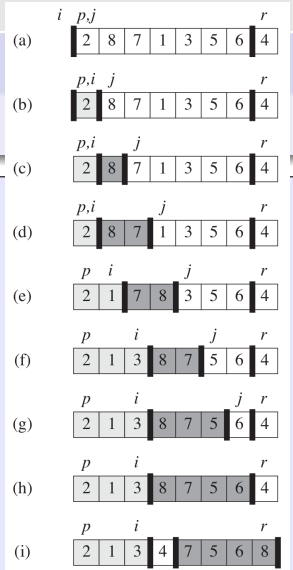
快速排序

划分的实现思想 (第二种思路)

从一个方向 (例如从左向右) 逐次扫描元素.




```
1  void
2  {
3    int
4    int
5
6    for (int j=p; j < r; ++j) {
7      if (A[j] <= x) {
8        ++i;
9        swap(A[i], A[j]);
10     }//if
11   }//for
12
13   swap(A[i+1], A[r]);
14
15   return i+1;
16 }
```



快速排序

时间复杂性分析

$$T_{best}(n) = \begin{cases} O(1) & n = 1 \\ 2T_{best}(n/2) + O(n) & n > 1 \end{cases}$$

故 $T_{best}(n) = O(n \log n)$.

$$T_{worst}(n) = T_{worse}(n-1) + O(n) = O(n^2).$$

$$T_{ave}(n) = \begin{cases} O(1) & n = 1 \\ \dfrac{1}{n-1} \displaystyle\sum_{i=1}^{n-1} [T_{ave}(i) + T_{ave}(n-i)] + O(n) & n > 1 \end{cases}$$

快速排序

快速排序算法的性能取决于划分的对称性，通过修改算法 partition，可以设计出采用随机选择策略的快速排序.

```
1 int randomized_partition(int* A, int p, int r)
2 {
3   int i = random(p,r);
4   swap(A[i], A[p]);
5   return partition(A, p, r);
6 }
```



线性时间选择

给定线性序集中 n 个元素和一个整数 k, $1 \leq k \leq n$, 要求找出这 n 个元素中第 k 小的元素.



线性时间选择

给定线性序集中 n 个元素和一个整数 k, $1 \leq k \leq n$, 要求找出这 n 个元素中第 k 小的元素.

```
1  // find the k-th element of A
2  int randomized_select(int* A, int p, int r, int k)
3  {
4    if (p<r) {
5      int q = randomized_partition(A, p, r);
6      int len = q-p+1;
7      if (len >= k) {
8        return randomized_select(A, p, q, k);
9      } else {
10       return randomized_select(A, q+1, r, k-len);
11     }
12   }
13 }
```



线性时间选择

给定线性序集中 n 个元素和一个整数 k, $1 \leq k \leq n$, 要求找出这 n 个元素中第 k 小的元素.

```
1  // find the k-th element of A
2  int randomized_select(int* A, int p, int r, int k)
3  {
4    if (p<r) {
5      int q = randomized_partition(A, p, r);
6      int len = q-p+1;
7      if (len >= k) {
8        return randomized_select(A, p, q,
9      } else {
10       return randomized_select(A, q+1,
11     }
12   }
13 }
```

类似快速排序, 只对其中一个部分进行递归处理.



线性时间选择

给定线性序集中 n 个元素和一个整数 k, $1 \le k \le n$, 要求找出这 n 个元素中第 k 小的元素

```
1  // find the k-th elem
2  int randomized_select
3  {
4    if (p<r) {
5      int q = randomize
6      int len = q-p+1;
7      if (len >= k) {
8        return randomiz
9      } else {
10       return randomized_select(A, q+1, r, k-len);
11     }
12   }
13 }
```

- 在最坏情况下, 算法 select 需要 $O(n^2)$ 计算时间.
- 但可以证明, 算法 select 可以在 $O(n)$ 平均时间内找出 n 个输入元素中的第 k 小元素.

线性时间选择

如果能在线性时间内找到一个划分基准，使得按这个基准所划分出的 2 个子数组的长度都至少为原数组长度的 ε 倍 ($0 < \varepsilon < 1$ 是某个正常数)，那么就可以在最坏情况下用 $O(n)$ 时间完成选择任务.



线性时间选择

如果能在线性时间内找到一个划分基准，使得按这个基准所划分出的 2 个子数组的长度都至少为原数组长度的 ε 倍 ($0 < \varepsilon < 1$ 是某个正常数)，那么就可以在最坏情况下用 $O(n)$ 时间完成选择任务.

例如，若 $\varepsilon = 9/10$，算法递归调用所产生的子数组的长度至少缩短 $1/10$. 所以，在最坏情况下，算法所需的计算时间 $T(n)$ 满足递归式 $T(n) \leq T(9n/10) + O(n)$. 由此可得 $T(n) = O(n)$.



线性时间选择

寻找划分基准

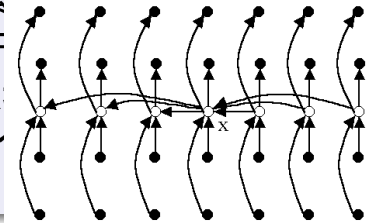
- 将 n 个输入元素划分成 $\lceil n/5 \rceil$ 个组, 每组 5 个元素, 只可能有一个组不是 5 个元素. 用任意一种排序算法, 将每组中的元素排好序, 并取出每组的中位数, 共 $\lceil n/5 \rceil$ 个.
- 递归调用 select 来找出这 $\lceil n/5 \rceil$ 个元素的中位数. 如果 $\lceil n/5 \rceil$ 是偶数, 就找它的 2 个中位数中较大的一个. 以这个元素作为划分基准.



线性时间选择

寻找划分基准

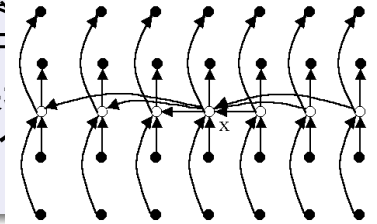
- 将 n 个输入元素划分成 $\lceil n/5 \rceil$ 个组, 每组 5 个元素, 只可能有一个组不是 5 个元素. 用任意一种排序算法, 将每组中的元素排好序, 并取出每组的中
- 递归调用 select 来 中位数. 如果 $\lceil n/5 \rceil$ 是偶数, 就找它的 2 . 以这个元素作为划分基准.



线性时间选择

寻找划分基准

- 将 n 个输入元素划分成 $\lceil n/5 \rceil$ 个组, 每组 5 个元素, 只可能有一个组不是 5 个元素. 用任意一种排序算法, 将每组中的元素排好序, 并取出每组的中
- 递归调用 select 来 中位数. 如果 $\lceil n/5 \rceil$ 是偶数, 就找它的 2 . 以这个元素作为划分基准.



设所有元素互不相同. 在这种情况下, 找出的基准 x 至少比 $3(n-5)/10$ 个元素大, 因为在每一组中有 2 个元素小于本组的中位数, 而 $n/5$ 个中位数中又有 $(n-5)/10$ 个小于基准 x. 同理, 基准 x 也至少比 $3(n-5)/10$ 个元素小. 而当 $n \geq 75$ 时, $3(n-5)/10 \geq n/4$, 所以按此基准划分所得的 2 个子数组的长度都至少缩短 $1/4$.

线性时间选择

复杂度分析

$$T(n) \le \begin{cases} O(1) & n < 75 \\ T(n/5) + T(3n/4) + O(n) & n \ge 75 \end{cases} \Rightarrow \mathrm{T(n)=O(n)}.$$

```
1  int select(int* A, int p, int r, int k)
2  {
3    if (r-p<75) {
4      用某个简单排序算法对数组A[p:r]排序;
5      return A[p+k-1];
6    }
7    for ( int i = 0; i<=(r-p-4)/5; ++i)
8    {将A[p+5*i]至A[p+5*i+4]的第3小元素与A[p+i]交换位置;}
9    //找中位数的中位数, r-p-4即上面所说的n-5
10   int x = select(A, p, p+(r-p-4)/5, (r-p-4)/10);
11   int i = partition(A,p,r,x); // 划分元素为x
12   int len=i-p+1;
13   if (k<=len) return select(A,p,i,k);
14   else return select(A,i+1,r,k-len);
15 }
```

线性时间选择

复杂度分析

$$T(n) \le \begin{cases} O(1) & n < 75 \\ T(n/5) + T(3n/4) + O(n) & n \ge 75 \end{cases} \Rightarrow \mathsf{T(n)=O(n)}.$$

```
1  int select(int* A, int p, int r, int k)
2  {
3    if (r-p<75) {
4      用某
5      retu
6    }
7    for (
8    {将A[p
9    //找中
10   int x
11   int i
12   int le
13   if (k<
14   else r
15 }
```

上述算法将每一组的大小定为 5, 并选取 75 作为是否作递归调用的分界点. 这 2 点保证了 $T(n)$ 的递归式中 2 个自变量之和 $n/5 + 3n/4 = 19n/20 = \varepsilon n$, $0 < \varepsilon < 1$. 这是使 $T(n) = O(n)$ 的关键之处. 当然, 除了 5 和 75 之外, 还有其他选择. 找中位数 x: $T(n/5)$, 按组找; 下次递归: $T(3/4n)$, 至少减少 $1/4$.

最接近点对问题

先来考虑一维的情形, 此时, S 中的 n 个点退化为 x 轴上的 n 个实数 $x_1, x_2, \ldots, x_n$, 最接近点对即为这 n 个实数中相差最小的 2 个实数.



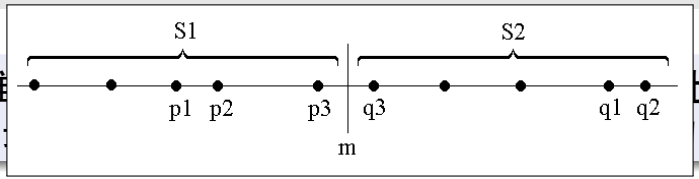
最接近点对问题

先来考虑一维的情形，此时，S 中的 n 个点退化为 x 轴上的 n 个实数 $x_1, x_2, \ldots, x_n$，最接近点对即为这 n 个实数中相差最小的 2 个实数.

- 假设用 x 轴上某个点 m 将 S 划分为 2 个子集 S_1 和 S_2，
- 基于平衡子问题的思想，用 S 中各点坐标的中位数来作分割点.
- 递归地在 S_1 和 S_2 上找出其最接近点对 $\{p_1, p_2\}$ 和 $\{q_1, q_2\}$，设 $d = \min\{|p_1 - p_2|, |q_1 - q_2|\}$.
- S 中的最接近点对：$\{p_1, p_2\}$，或者 $\{q_1, q_2\}$，或者 $\{p_3, q_3\}$，其中 $p_3 \in S_1, q_3 \in S_2$. 能否在线性时间内找到 p_3, q_3?



最接近点对问题

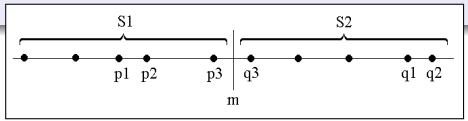


先来考虑一维 上的 n 个实数 $x_1, x_2, \ldots, x_n$, 2 个实数.

- 假设用 x 轴上某个点 m 将 S 划分为 2 个子集 S_1 和 S_2,
- 基于平衡子问题的思想, 用 S 中各点坐标的中位数来作分割点.
- 递归地在 S_1 和 S_2 上找出其最接近点对 $\{p_1, p_2\}$ 和 $\{q_1, q_2\}$, 设 $d = \min\{|p_1 - p_2|, |q_1 - q_2|\}$.
- S 中的最接近点对: $\{p_1, p_2\}$, 或者 $\{q_1, q_2\}$, 或者 $\{p_3, q_3\}$, 其中 $p_3 \in S_1, q_3 \in S_2$. 能否在线性时间内找到 p_3, q_3?

最接近点对问题

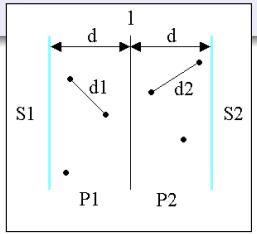
- 如果 S 的最接近点对是 $\{p_3, q_3\}$, 即 $|p_3 - q_3| < d$, 则 p_3 和 q_3 两者与 m 的距离不超过 d, 即 $p_3 \in (m-d, m]$, $q_3 \in (m, m+d]$.
- 由于在 S_1 中, 每个长度为 d 的半闭区间至多包含一个点（否则必有两点且距离小于 d）, 并且 m 是 S_1 和 S_2 的分割点, 因此 $(m-d, m]$ 中至多包含 S 中的一个点. 由图可以看出, 如果 $(m-d, m]$ 中有 S 中的点, 则此点就是 S_1 中最大点.
- 因此, 用线性时间就能找到区间 $(m-d, m]$ 和 $(m, m+d]$ 中所有点, 即 p_3 和 q_3. 从而我们用线性时间就可以将 S_1 的解和 S_2 的解合并成为 S 的解.



最接近点对问题

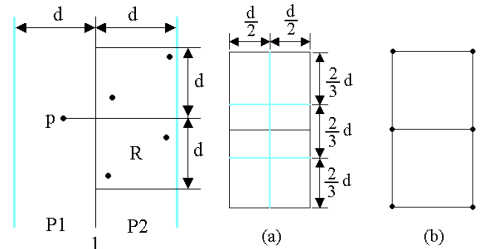
下面来考虑二维的情形.

- 选取一垂直线 $\ell : x = m$ 来作为分割直线, 其中 m 为 S 中各点 x 坐标的中位数, 由此将 S 分割为 S_1 和 S_2.
- 递归地在 S_1 和 S_2 上找出其最小距离 d_1 和 d_2，并设 $d = \min\{d_1, d_2\}$，S 中的最接近点对或者是 d, 或者是某个 $\{p, q\}$，其中 $p \in P_1$, $q \in P_2$ (P_1, P_2 如图所示).
- 能否在线性时间内找到 p, q?



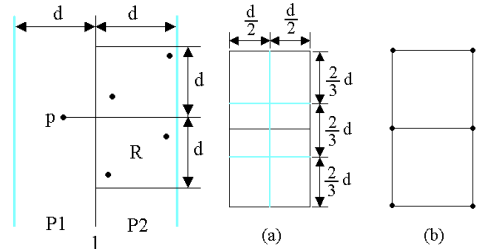
最接近点对问题

- 考虑 P_1 中任意一点 p, 它若与 P_2 中的点 q 构成最接近点对的候选者, 则必有 $distance(p, q) < d$, 满足这个条件的 P_2 中的点 q 一定落在一个 $d \times 2d$ 的矩形 R 中;
- 由 d 的意义可知, P_2 中任何 2 个 S 中的点的距离都不小于 d, 由此可以推出矩形 R 中最多只有 6 个 S 中的点;
- 因此, 在分治法合并步骤中最多只需检查 $6n/2 = 3n$ 个候选者.



最接近点对问题

- 考虑 P_1 中任意一点 p, 它若与 P_2 中的点 q 构成最接近点对的候选者, 则必有 $distance(p, q) < d$, 满足这个条件的 P_2 中的点 q 一定落在一个 $d \times 2d$ 的矩形 R 中;
- 由 d 的意义可知, P_2 中任何 2 个 S 中的点的距离都不小于 d, 由此可以推出矩形 R 中最多只有 6 个 S 中的点;
- 因此, 在分治法合并步骤中最多只需检查 $6n/2 = 3n$ 个候选者.



证明: 将矩形 R 的长为 $2d$ 的边 3 等分, 将它的长为 d 的边 2 等分, 由此导出 6 个 $(d/2) \times (2d/3)$ 的矩形. 若矩形 R 中有多于 6 个 S 中的点, 则由鸽巢原理易知至少有一个 $(d/2) \times (2d/3)$ 的小矩形中有 2 个以上 S 中的点. 设 u, v 是位于同一小矩形中的 2 个点, 则

$$(x(u) - x(v))^2 + (y(u) - y(v))^2 \le (d/2)^2 + (2d/3)^2 = \frac{25}{36} d^2.$$

$distance(u, v) < d$. 这与 d 的意义相矛盾.

最接近点对问题

- 为了确切地知道要检查哪 6 个点，可以将 p 和 P_2 中所有 S_2 的点投影到垂直线 ℓ 上. 由于能与 p 点一起构成最接近点对候选者的 S_2 中点一定在矩形 R 中, 所以它们在直线 ℓ 上的投影点距离 p 在 ℓ 上投影点的距离小于 d. 由上面的分析可知, 这种投影点最多只有 6 个.
- 因此, 若将 P_1 和 P_2 中所有 S 中点按其 y 坐标排好序, 则对 P_1 中所有点, 对排好序的点列作一次扫描, 就可以找出所有最接近点对的候选者. 对 P_1 中每一点最多只要检查 P_2 中排好序的相继 6 个点.



最接近点对问题

- 为了确切地知道要检查哪 6 个点，可以将 p 和 P_2 中所有 S_2 的点投影到垂直线 ℓ 上. 由于能与 p 点一起构成最接近点对候选者的 S_2 中点一定在矩形 R 中，所以它们在直线 ℓ 上的投影点距离
- 因

复杂度分析:

$$T(n) = \begin{cases} O(1) & n < 4 \\ 2T(n/2) + O(n) & n \geq 4 \end{cases}$$

故 $T(n) = O(n \log n)$. (另外，排序时间也是 $O(n \log n)$)

循环赛日程表

设计一个满足以下要求的比赛日程表:

1) 每个选手必须与其他 $n-1$ 个选手各赛一次;
2) 每个选手一天只能赛一次;
3) 循环赛一共进行 $n-1$ 天.

1	2	3	4	5	6	7	8
2	1	4	3	6	5	8	7
3	4	1	2	7	8	5	6
4	3	2	1	8	7	6	5
5	6	7	8	1	2	3	4
6	5	8	7	2	1	4	3
7	8	5	6	3	4	1	2
8	7	6	5	4	3	2	1

循环赛日程表

设计一个满足以下要求的比赛日程表:

1) 每个选手必须与其他 $n-1$ 个选手各赛一次;
2) 每个选手一天只能赛一次;
3) 循环赛一共进行 $n-1$ 天.

1	2	3	4	5	6	7	8
2	1	4	3	6	5	8	7
3	4	1	2	7	8	5	6
4	3	2	1	8	7	6	5
5	6	7	8	1	2	3	4
6	5	8	7	2	1	4	3
7	8	5	6	3	4	1	2
8	7	6	5	4	3	2	1

按分治策略, 将所有的选手分为两半, n 个选手的比赛日程表就可以通过为 $n/2$ 个选手设计的比赛日程表来决定. 递归地用对选手进行分割, 直到只剩下 2 个选手时, 比赛日程表的制定就变得很简单. 这时只要让这 2 个选手进行比赛就可以了.

